Python

Task	Command	Notes
Version manager	`uv`	`uv python install 3.12` to install a version
New project	`uv init name`	`--lib` for library; creates `pyproject.toml`
Add dependency	`uv add pkg`	`--dev` for dev-only dependency
Install deps	`uv sync`	installs from `uv.lock`
Build	`uv build`	builds wheel + sdist to `dist/`
Run	`uv run script.py`	runs in project virtualenv
REPL	`uv run python` / `ipython`	
Test	`uv run pytest`	`-x` stop first fail, `-k pat` filter
Format	`ruff format .`	`--check` for dry run
Lint	`ruff check .`	`--fix` to autofix
LSP	`pyright`	started by nvim and Emacs eglot

uv Extras

`uv remove pkg`	Remove dependency
`uv lock`	Update lockfile
`uv tool install pkg`	Install global CLI
`uv tool list`	List global tools
`uvx cmd`	Run tool without install
`uv python list`	Available versions
`uv python pin 3.12`	Pin project version

Common Patterns

`[x for x in xs if p(x)]`	List comprehension
`{k: v for k, v in d}`	Dict comprehension
`x if cond else y`	Ternary
`a, b = b, a`	Swap
`*args, **kwargs`	Variadic args
`yield`	Generator
`with open(f) as fh:`	Context manager
`match x:`	Pattern match (3.10+)
`case pat:`	Match arm
`f"{x:.2f}"`	f-string formatting

Error Handling

`try: ... except E as e:`	Catch exception
`except (A, B):`	Catch multiple types
`else:`	No exception branch
`finally:`	Always runs
`raise ValueError("msg")`	Raise exception
`raise X from e`	Chain exceptions
`class E(Exception): ...`	Custom exception

Strings & Slicing

`" ".join(xs)`	Join with separator
`s.split(",")`	Split string
`s.strip()`	Trim whitespace
`s.startswith("x")`	Prefix test
`s.replace(a, b)`	Replace substring
`xs[1:-1]`	Slice (drop first/last)
`xs[::2]`	Every 2nd element
`xs[::-1]`	Reverse

Type Hints

`x: int = 0`	Variable annotation
`def f(x: str) -> bool:`	Function signature
`list[int]`	Generic list
`dict[str, Any]`	Generic dict
`X | None`	Optional (3.10+)
`TypeAlias = ...`	Type alias
`@dataclass`	Data class
`Protocol`	Structural typing
`# pyright: strict`	Strict mode (file)

Standard Library

`pathlib`	File paths
`itertools`	chain, product, groupby
`functools`	lru_cache, partial, reduce
`collections`	Counter, defaultdict, deque
`dataclasses`	@dataclass
`typing`	Type hint utilities
`json`	JSON encode/decode
`subprocess`	Run shell commands
`argparse`	CLI argument parsing
`logging`	Structured logging

Key Libraries

`polars`	Fast DataFrames
`numpy`	Array computing
`scipy`	Numerics, optimization
`matplotlib`	Plotting
`xarray`	N-dim labeled arrays
`geopandas`	Geospatial DataFrames
`httpx`	HTTP client (async)
`pydantic`	Validation / settings
`pytest`	Testing framework

Decorators

`@property`	Getter method
`@x.setter`	Setter method
`@staticmethod`	No self/cls arg
`@classmethod`	Receives cls arg
`@abstractmethod`	Must override (ABC)
`def dec(f): ...`	Custom
`return w`	decorator
`@wraps(f)`	Preserve metadata

Async

`async def f():`	Async function
`await coro`	Await coroutine
`asyncio.run(main())`	
Run event loop	
`asyncio.gather(*coros)`	
Run concurrently	
`async with`	Async context manager
`async for x in ait:`	Async iteration

Data Tools

`duckdb`	SQL REPL (parquet/CSV)
`vd file`	VisiData TUI
`quarto render f.qmd`	Render document

Testing Patterns

`def test_name():`	Test function
`assert x == y`	Assertion
`@pytest.fixture`	Fixture decorator
`@pytest.mark.parametrize`	
Parametrize tests	
`pytest.raises(Exc)`	Expect exception
`tmp_path`	Built-in temp dir fixture

Functional Patterns

`map(f, xs)`	Apply f to each element
`filter(f, xs)`	Keep elements where f is true
`functools.reduce(f, xs)`	Fold / accumulate
`functools.partial(f, a)`	Partial application
`@functools.lru_cache`	Memoization
`lambda x:` *expr*	Anonymous function
`zip(xs, ys)`	Pair elements
`enumerate(xs)`	Index + element
`any(p(x) for x in xs)`	Existential
`all(p(x) for x in xs)`	Universal
`sorted(xs, key=f)`	Sort by projection
`itertools.chain(a, b)`	Concatenate iterables
`itertools.product(a, b)`	Cartesian product
`itertools.starmap(f, pairs)`	Map unpacking args
`more-itertools`	Extended combinators (pip)

FP LIBRARIES

`toolz / cytoolz`	Composable FP: pipe, curry, merge
`returns`	Result, Maybe, IO monads
`expression`	Pipe, effect system, Result
`more-itertools`	200+ iterator recipes

Python — Scientific & Geospatial

NumPy — Creation

`np.array([1, 2, 3])`	From list
`np.zeros((m, n))`	Zero matrix
`np.ones((m, n))`	Ones matrix
`np.eye(n)`	Identity matrix
`np.arange(0, 10, 0.5)`	Range with step
`np.linspace(0, 1, 100)`	Evenly spaced
`np.random.default_rng()`	RNG instance
`rng.normal(0, 1, (m, n))`	Normal samples
`np.full((m, n), v)`	Fill with value

NumPy — Indexing & Shape

`a[1, 2]`	Element access
`a[1:3, :]`	Row slice
`a[:, 0]`	Column slice
`a[a > 0]`	Boolean indexing
`a.shape`	Dimensions tuple
`a.reshape(m, n)`	Reshape
`a.T`	Transpose
`a.flatten()`	1-D copy
`np.concatenate([a, b])`	Join arrays
`np.stack([a, b])`	Stack new axis

NumPy — Operations

`a + b, a * b`	Elementwise ops
`a @ b`	Matrix multiply
`np.dot(a, b)`	Dot product
`a.sum(axis=0)`	Sum along axis
`a.mean(), a.std()`	Statistics
`a.min(), a.max()`	Min / max
`np.argmax(a)`	Index of max
`np.where(cond, x, y)`	Conditional select
`np.sort(a)`	Sort array
`np.unique(a)`	Unique values

NumPy — Linear Algebra

`np.linalg.inv(A)`	Inverse
`np.linalg.det(A)`	Determinant
`np.linalg.eig(A)`	Eigenvalues, vectors
`np.linalg.svd(A)`	SVD
`np.linalg.solve(A, b)`	Solve Ax = b
`np.linalg.norm(x)`	Vector/matrix norm
`np.linalg.lstsq(A, b)`	Least squares

Pandas — DataFrame

`pd.read_csv(f)`	Read CSV
`pd.read_parquet(f)`	Read Parquet
`df.to_csv(f)`	Write CSV
`df.head(n)`	First n rows
`df.info()`	Column types + nulls
`df.describe()`	Summary stats
`df.shape`	(rows, cols)
`df.columns`	Column names
`df.dtypes`	Column types

Pandas — Selection

`df["col"]`	Column (Series)
`df[["a", "b"]]`	Multiple columns
`df.loc[row, col]`	Label-based
`df.iloc[i, j]`	Position-based
`df[df.x > 0]`	Boolean filter
`df.query("x > 0")`	Query string
`df.sort_values("col")`	Sort by column
`df.drop_duplicates()`	Remove duplicates
`df.isna()`	Null check
`df.fillna(v)`	Fill nulls

Pandas — Transform

`df.groupby("k").agg(f)`
Group + aggregate

`df.pivot_table(...)`	Pivot / crosstab
`pd.merge(a, b, on="k")`	Join DataFrames
`pd.concat([a, b])`	Stack DataFrames
`df.apply(f)`	Apply function
`df.assign(y=lambda d: ...)`	Add column

`df.rename(columns={...})`
Rename columns

`df.melt(...)`	Wide to long

Math Module

`math.sqrt(x)`	Square root
`math.sin, cos, tan`	Trig functions
`math.log(x)` / `log10`	Ln / log base 10
`math.exp(x)`	e^x
`math.pi, math.e`	Constants
`math.ceil, math.floor`	Round up / down
`math.gcd(a, b)`	GCD
`math.comb(n, k)`	Binomial coeff
`math.isclose(a, b)`	Float comparison

Shapely — Geometry

`Point(x, y)`	Point geometry
`LineString([(x,y), ...])`	Line geometry
`Polygon([(x,y), ...])`	Polygon geometry
`g.area, g.length`	Measurements
`g.buffer(d)`	Buffer distance
`g.intersection(h)`	Intersection
`g.union(h)`	Union
`g.contains(h)`	Containment test
`g.distance(h)`	Distance between
`g.centroid`	Centroid point

GeoPandas

`gpd.read_file(f)`	Read shapefile/gpkg
`gpd.read_parquet(f)`	Read GeoParquet
`gdf.to_file(f)`	Write to file
`gdf.geometry`	Geometry column
`gdf.crs`	Coordinate ref system
`gdf.to_crs(epsg=4326)`	Reproject
`gdf.plot()`	Quick plot
`gpd.sjoin(a, b)`	Spatial join

`gdf.dissolve(by="col")`
Merge geometries

`gdf.overlay(b, how="...")`	Spatial overlay
`gdf.clip(mask)`	Clip to boundary

Rasterio

`rasterio.open(f)`	Open raster file
`src.read(1)`	Read band 1 (array)
`src.bounds`	Spatial extent
`src.crs`	Coordinate ref system
`src.transform`	Affine transform
`src.meta`	Metadata dict

`rasterio.plot.show(src)`
Quick plot

Coordinate Reference

`EPSG:4326`	WGS 84 (lon/lat)
`EPSG:3857`	Web Mercator
`pyproj.Transformer`	CRS transforms
`t.transform(x, y)`	Transform coords